

Foundations of Gradient descent

[Gradient descent]

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

1.0 Content Block

$G(\mathbf{x}) = \begin{bmatrix} 3x_1 - \cos(x_2 x_3) - 324x_1^2 - 625x_2^2 + 2x_2 - 1 \exp(-x_1 x_2) + 20x_3 + 10\pi - 3 \\ 4x_1^2 - 625x_2^2 + 2x_2 - 1 \exp(-x_1 x_2) + 20x_3 + \frac{10\pi - 3}{3} \end{bmatrix}$, $\displaystyle G(\mathbf{x}) = \begin{bmatrix} 3x_1 - \cos(x_2 x_3) - \frac{1}{3} \end{bmatrix}$

2.0 Content Block

Gradient descent is based on the observation that if the multi-variable function $f(\mathbf{x})$ is defined and differentiable in a neighborhood of a point $\mathbf{a}$, then $f(\mathbf{x})$ decreases fastest if one goes from $\mathbf{a}$ in the direction of the negative gradient of f at $\mathbf{a}$, i.e., $-\nabla f(\mathbf{a})$. It follows that, if

3.0 Content Block

Further reading Boyd, Stephen; Vandenberghe, Lieven (2004). "Unconstrained Minimization" (PDF). Convex Optimization. New York: Cambridge University Press. pp. 457–520. ISBN 0-521-83378-7. Chong, Edwin K. P.; Jak, Stanislaw H. (2013). "Gradient Methods". An Introduction to Optimization (Fourth ed.). Hoboken: Wiley. pp. 131–160. ISBN 978-1-118-27901-4. Himmelblau, David M. (1972). "Unconstrained Minimization Procedures Using Derivatives". Applied Nonlinear Programming. New York: McGraw-Hill. pp. 63–132. ISBN 0-07-028921-2.

4.0 Content Block

for a small enough step size or learning rate $\eta \in \mathbb{R}_+$, then $f(\mathbf{a}_n) \geq f(\mathbf{a}_{n+1})$. In other words, the term $\eta \nabla f(\mathbf{a})$ is subtracted from $\mathbf{a}$ because we want to move against the gradient, toward the local minimum. With this observation in mind, one starts with a guess $\mathbf{x}_0$ for a local minimum of f , and considers the sequence $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$ such that

5.0 Content Block

The idea is to take repeated steps in the opposite direction of the gradient (or approximate gradient) of the function at the current point, because this is the direction of steepest descent. Conversely, stepping in the direction of the gradient will lead to a trajectory that maximizes that function; the procedure is then known as gradient ascent. Gradient descent should not be confused with local search algorithms, although both are iterative methods for optimization. Gradient descent is particularly useful in machine learning and artificial intelligence for minimizing the cost or loss function. Gradient descent is generally attributed to Augustin-Louis Cauchy, who first suggested it in 1847. Jacques Hadamard independently proposed a similar method in 1907. Its convergence properties for non-linear optimization problems were first studied by Haskell Curry in 1944, with the method becoming increasingly well-studied and used in the following decades. A simple extension of gradient descent, stochastic gradient descent, serves as the most basic

algorithm used for training most deep networks today.

6.0 Content Block

so the sequence $(\mathbf{x}_n)$ converges to the desired local minimum. Note that the value of the step size η is allowed to change at every iteration. It is possible to guarantee the convergence to a local minimum under certain assumptions on the function f (for example, f convex and ∇f Lipschitz) and particular choices of η . Those include the sequence

7.0 Content Block

$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \dots$,

8.0 Content Block

Because each step is taken in the steepest direction, steepest-descent steps alternate between directions aligned with the extreme axes of the elongated level sets. When $\kappa(\mathbf{A})$ is large, this produces a characteristic zig-zag path. The poor conditioning of $\mathbf{A}$ is the primary cause of the slow convergence, and orthogonality of successive residuals reinforces this alternation. As shown in the image on the right, steepest descent converges slowly due to the high condition number of $\mathbf{A}$, and the orthogonality of residuals forces each new direction to undo the overshoot from the previous step. The result is a path that zigzags toward the solution. This inefficiency is one reason conjugate gradient or preconditioning methods are preferred.

9.0 Content Block

To avoid multiplying by $\mathbf{A}$ twice per iteration, we note that $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$ implies $\mathbf{r} := \mathbf{r} - \eta \mathbf{A} \mathbf{r}$, which gives the traditional algorithm,

10.0 Content Block

$\mathbf{r} := \mathbf{b} - \mathbf{A} \mathbf{x}$ repeat in the loop: $\eta := \mathbf{r}^T \mathbf{r} / \mathbf{r}^T \mathbf{A} \mathbf{r}$ $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$ if $\mathbf{r}^T \mathbf{r}$ is sufficiently small, then exit loop $\mathbf{r} := \mathbf{r} - \eta \mathbf{A} \mathbf{r}$ end repeat loop return $\mathbf{x}$ as the result

11.0 Content Block

$\mathbf{x}^{(1)} = \mathbf{0} - \eta_0 \nabla f(\mathbf{0}) = \mathbf{0} - \eta_0 \mathbf{J}_G(\mathbf{0}) \mathbf{G}(\mathbf{0})$, $\mathbf{x}^{(j+1)} = \mathbf{x}^{(j)} - \eta_j \nabla f(\mathbf{x}^{(j)}) = \mathbf{x}^{(j)} - \eta_j \mathbf{J}_G(\mathbf{x}^{(j)}) \mathbf{G}(\mathbf{x}^{(j)})$,

12.0 Content Block

Using gradient descent in C++, Boost, Ublas for linear regression Series of Khan Academy videos discusses gradient ascent Online book teaching gradient descent in deep neural network context Archived at Ghostarchive and the Wayback Machine: "Gradient Descent, How Neural Networks Learn". 3Blue1Brown. October 16, 2017 – via YouTube. Garrigos, Guillaume; Gower, Robert M. (2023). "Handbook of Convergence Theorems for (Stochastic) Gradient Methods". arXiv:2301.11235 [math.OC].